

Linux USB Mass Storage Device Driver Documentation

Field	Details
Project Title	Linux USB Mass Storage Device Driver
Team	P1 Team 1
Author	A Prudhvi
Platform	Linux Kernel — C Programming
Driver Type	USB Kernel Module (usb_device_driver.ko)
Protocol	USB Bulk-Only Transport + SCSI
Block Interface	/dev/myusb0 via blk-mq
Tools	Raspberry pi 4, Transcend Card reader ,micro sd card

1. Abstract

This document describes the design, implementation, and testing of a custom Linux kernel USB Mass Storage device driver written entirely in C. The driver implements the USB Bulk-Only Transport (BOT) protocol and communicates with a USB flash drive using a subset of SCSI commands: INQUIRY, TEST UNIT READY, REQUEST SENSE, READ CAPACITY(10), READ(10), and WRITE(10).

The driver registers a standard Linux block device (/dev/myusb0) using the blk-mq multi-queue framework, allowing the device to be mounted and accessed with ordinary Linux tools such as mount, dd, and fdisk. The driver correctly handles device probe, disconnect, error recovery, and mass-storage reset. Measured performance on real hardware reached 34 MB/s write and 97 MB/s read.

2. Introduction

USB Mass Storage is a standard protocol that allows computers to communicate with external storage devices such as flash drives, hard disks, and SD card readers over USB. It uses familiar SCSI commands packaged inside a simple transport protocol called Bulk-Only Transport (BOT).

Linux already includes a built-in USB storage driver (usb-storage). This project builds a new, independent driver from scratch to demonstrate a complete understanding of how the Linux kernel interfaces with USB hardware at the driver level. No part of the existing usb-storage driver is reused.

The driver is implemented as a loadable kernel module, which means it can be inserted into the running kernel without rebooting, tested, and removed cleanly.

3. Objective

The primary objectives of this project are:

- Write a Linux kernel module in C that binds to any USB Mass Storage device automatically.
- Implement the full USB Bulk-Only Transport pipeline: CBW → Data → CSW.
- Send real SCSI commands to the device to identify it, check readiness, and read its capacity.
- Register a standard Linux block device (/dev/myusbX) using the blk-mq framework.
- Handle read and write I/O requests from the block layer and translate them into SCSI READ(10) and WRITE(10) commands sent over USB.
- Implement error handling, endpoint halt recovery, mass-storage reset, and clean disconnect handling.
- Validate the driver by mounting the device and performing read/write tests with dd.

4. System Architecture

The driver sits between the Linux block layer and the USB hardware. The following diagram shows how a user read or write request travels through the kernel layers to reach the physical device:

Layer	Component
User Space	Application (e.g., cp, dd, mount)
Kernel — VFS	Virtual File System — file abstraction
Kernel — Block	Block Layer / blk-mq — request queue
Our Driver	my_usb_mass_storage — queue_rq()
SCSI Layer	SCSI commands (READ10, WRITE10, etc.)
Transport	USB Bulk-Only Transport (CBW/CSW)
Hardware	USB Flash Drive

The central data structure is struct my_usb, which holds the USB device reference, bulk endpoint addresses, SCSI parameters (last LBA, block size, capacity), the gendisk and blk_mq_tag_set for the block device, a mutex protecting USB I/O, and an atomic disconnect flag.

5. USB Mass Storage Protocol

5.1 Bulk-Only Transport Overview

The Bulk-Only Transport (BOT) protocol uses two USB bulk endpoints — one IN (device to host) and one OUT (host to device) — to carry all data. There are no interrupt or isochronous endpoints. Every operation follows a strict three-phase sequence:

Phase	Description
1. CBW	Host sends a 31-byte Command Block Wrapper containing the SCSI command.
2. Data	Data is transferred: host sends data (OUT) for writes, device sends data (IN) for reads. This phase is absent for commands with no data (e.g., TEST UNIT READY).
3. CSW	Device returns a 13-byte Command Status Wrapper indicating pass, fail, or phase error.

5.2 Command Block Wrapper (CBW)

The CBW is a fixed 31-byte structure sent on the Bulk OUT endpoint. Key fields:

- Signature: always 0x43425355 ("USBC" in little-endian)
- Tag: a unique 32-bit identifier matched by the CSW
- Data Transfer Length: number of bytes expected in the data phase
- Flags: 0x80 = data IN (read), 0x00 = data OUT (write)
- LUN: Logical Unit Number (always 0 in this driver)
- CDB Length + CDB: SCSI command bytes (6 or 10 bytes)

5.3 Command Status Wrapper (CSW)

The CSW is a fixed 13-byte structure received on the Bulk IN endpoint. Key fields:

- Signature: always 0x53425355 ("USBS")
- Tag: must match the CBW tag exactly
- Data Residue: bytes not transferred
- Status: 0x00 = Pass, 0x01 = Fail, 0x02 = Phase Error (triggers reset)

6. SCSI Commands

The driver uses the following SCSI commands. Each is encapsulated in a CBW and sent over USB:

Command (Opcode)	Purpose
INQUIRY (0x12)	Retrieves device identity: vendor name, product name, and firmware revision. Called once during probe. Implemented in <code>scsi_cmd_inquiry()</code> .
TEST UNIT READY (0x00)	Checks whether the device is ready to accept read/write commands. Called after INQUIRY. Implemented in <code>scsi_cmd_test_unit_ready()</code> .
REQUEST SENSE (0x03)	Reads 18 bytes of error information when TEST UNIT READY fails. Implemented in <code>usb_mass_storage_request_sense()</code> .
READ CAPACITY(10) (0x25)	Returns the last Logical Block Address (LBA) and block size. Used to calculate total storage capacity. Implemented in <code>scsi_cmd_read_capacity()</code> .
READ(10) (0x28)	Reads one or more 512-byte blocks from a given LBA. Called by <code>queue_rq()</code> for every read request. Implemented in <code>scsi_cmd_read10()</code> .
WRITE(10) (0x2A)	Writes one or more blocks to a given LBA. Called by <code>queue_rq()</code> for every write request. Implemented in <code>scsi_cmd_write10()</code> .

7. Driver Implementation

7.1 Module Registration

The driver registers itself with the USB subsystem using `usb_register()` in `usb_mass_storage_init()`. The device ID table uses `USB_INTERFACE_INFO()` to match any device with class 0x08 (Mass Storage), subclass 0x06 (SCSI Transparent), and protocol 0x50 (Bulk-Only Transport). This ensures the driver binds to any compliant USB flash drive automatically.

A block device major number is obtained at init time using `register_blkdev(0, "usb_ms")`, which lets the kernel assign a free major number dynamically.

7.2 Probe Function

When a matching USB device is connected, the kernel calls `usb_mass_storage_probe()`. The probe function:

- Allocates the `my_usb` structure with `kzalloc()`
- Calls `endpoints_discovery()` to find the Bulk IN and Bulk OUT endpoint addresses by scanning the interface descriptor
- Sends INQUIRY, TEST UNIT READY (with REQUEST SENSE on failure), and READ CAPACITY SCSI commands
- Calls `usb_mass_storage_setup_block_device()` to create the block device

7.3 USB Transfer Functions

All USB transfers use `usb_bulk_msg()`, a synchronous (blocking) USB transfer function with a 5-second timeout. Three internal helper functions wrap it:

- `submit_cbw()` — sends the CBW on the Bulk OUT pipe
- `submit_data_in()` — receives data on the Bulk IN pipe (read operations)
- `submit_data_out()` — sends data on the Bulk OUT pipe (write operations)
- `submit_csw()` — receives the CSW on the Bulk IN pipe; retries once on EPIPE, then triggers mass-storage reset on total failure

Endpoint halt conditions (EPIPE error) are cleared with `usb_clear_halt()` before retrying.

7.4 `build_scsi_command()`

All six SCSI command functions share a single builder: `build_scsi_command()`. It allocates CBW and CSW structures, fills in the CBW fields (signature, tag, data length, direction flags, CDB), then calls `submit_cbw()`, `submit_data_in()` or `submit_data_out()` as appropriate, and finally `submit_csw()` to validate the response. Memory is freed on both success and failure paths.

8. Block Device and blk-mq

8.1 Setup

The function `usb_mass_storage_setup_block_device()` configures the Linux block device layer:

- A `blk_mq_tag_set` is configured with 1 hardware queue, queue depth 1, and `BLK_MQ_F_BLOCKING` (requests may sleep — required since USB I/O can block).
- `blk_mq_alloc_tag_set()` allocates the tag set.
- `blk_mq_alloc_disk()` creates the gendisk and associated request queue.
- The disk is named `myusb%d` (e.g., `myusb0`) using `snprintf()` and the per-device index from the atomic counter `dev_index`.
- `set_capacity()` sets the disk size in 512-byte sectors: `sectors = (last_lba + 1) × block_size / 512`.
- `device_add_disk()` registers the disk with the kernel, making `/dev/myusb0` appear.

8.2 `queue_rq()` — Request Handler

Every block I/O request from the kernel arrives at `usb_ms_queue_rq()`. It:

- Calls `blk_mq_start_request()` to mark the request as started.
- Validates the request type (READ or WRITE; FLUSH is silently completed; others return `BLK_STS_NOTSUPP`).
- Converts the block layer sector number (in 512-byte units) to a SCSI LBA: `lba = (sector × 512) / block_size`.
- Allocates a kernel buffer with `kmalloc()` for the data.
- Acquires `io_mutex` to serialize USB operations.
- For WRITE: calls `copy_from_request()` to gather data from the bio, then `scsi_cmd_write10()`.
- For READ: calls `scsi_cmd_read10()`, then `copy_to_request()` to scatter data back into the bio.
- Calls `blk_mq_end_request()` with `BLK_STS_OK` or `BLK_STS_IOERR` to complete the request.

8.3 Page Mapping

Linux I/O requests carry data in page-mapped segments. `copy_from_request()` and `copy_to_request()` iterate over all segments using the `rq_for_each_segment` macro. Each segment page is temporarily mapped into the kernel address space using `kmap_local_page()`, data is copied with `memcpy()`, and the mapping is released with `kunmap_local()`.

9. Read and Write Operations

9.1 Read Operation

Complete flow from application to hardware:

Step	Action
1	Application calls read() → VFS → block layer creates a request
2	blk-mq calls usb_ms_queue_rq()
3	$LBA = \text{sector} \times 512 / \text{block_size}$
4	scsi_cmd_read10(dev, lba, blocks, buffer) called
5	CBW built with opcode 0x28, LBA, and block count; sent via Bulk OUT
6	Data received from device via Bulk IN into kernel buffer
7	CSW received and validated (status must be 0x00)
8	copy_to_request() copies buffer into bio pages
9	blk_mq_end_request(rq, BLK_STS_OK)

9.2 Write Operation

Step	Action
1	Application calls write() → VFS → block layer creates a request
2	blk-mq calls usb_ms_queue_rq()
3	copy_from_request() gathers data from bio pages into kernel buffer
4	scsi_cmd_write10(dev, lba, blocks, buffer) called
5	CBW built with opcode 0x2A, LBA, and block count; sent via Bulk OUT
6	Data buffer sent to device via Bulk OUT
7	CSW received and validated
8	blk_mq_end_request(rq, BLK_STS_OK)

10. Error Handling

The driver implements multi-level error handling to remain stable under abnormal conditions:

Error Condition	Handling
USB Bulk Transfer Failure	Returns -EIO. On EPIPE (endpoint stall), usb_clear_halt() is called to clear the endpoint before retrying.

CSW Signature Mismatch	Expected value is 0x53425355. Any other value returns -EIO immediately.
CSW Tag Mismatch	The CSW tag must equal the CBW tag. Mismatch returns -EIO.
CSW Status FAIL (0x01)	Returns -EIO; the upper layer may retry the request.
CSW Status Phase Error (0x02)	Triggers <code>usb_mass_storage_reset_recovery()</code> : sends the USB Mass Storage Reset control request and clears both endpoints.
CSW Receive Failure	Retried once; if second attempt also fails, full reset recovery is triggered.
Invalid LBA	<code>scsi_cmd_read10()</code> and <code>scsi_cmd_write10()</code> check <code>(lba + blocks)</code> against <code>last_lba + 1</code> and return -EINVAL if exceeded.
Invalid Block Size	A <code>block_size</code> of 0 returned by READ CAPACITY causes probe to fail with -EIO.
Device Disconnected	<code>atomic_t disconnected</code> is set to 1 in <code>disconnect()</code> . All new I/O in <code>queue_rq()</code> and <code>open()</code> returns immediately with <code>BLK_STS_IOERR</code> or <code>-ENODEV</code> .

11. Driver Disconnect

When the USB device is unplugged, the kernel calls `usb_mass_storage_disconnect()`:

- `atomic_set(&dev->disconnected, 1)` — prevents new I/O from starting.
- `mutex_lock(&dev->io_mutex)` — waits for any in-progress I/O to finish.
- `usb_ms_remove_disk()` calls `del_gendisk()` and `put_disk()` to remove the block device from the kernel.
- `blk_mq_free_tag_set()` frees the blk-mq tag set.
- `usb_put_dev()` releases the USB device reference.
- `kfree(dev)` frees the `my_usb` structure.

This sequence ensures no use-after-free occurs and that the block device is cleanly removed before memory is released.

12. Important Kernel Functions

Function	Purpose
<code>usb_bulk_msg()</code>	Synchronous USB bulk transfer — sends or receives data on a bulk endpoint with timeout.
<code>usb_control_msg()</code>	Sends a USB control request — used for Mass Storage Reset.

usb_sndbulkpipe()	Creates a Bulk OUT pipe handle for the given endpoint address.
usb_rcvbulkpipe()	Creates a Bulk IN pipe handle for the given endpoint address.
usb_clear_halt()	Clears an endpoint stall (HALT condition) — sent after EPIPE errors.
kmalloc() / kzalloc()	Allocates kernel memory. kzalloc() zeros it. Used for CBW, CSW, and data buffers.
kfree()	Frees kernel-allocated memory. Called on all code paths after use.
kmap_local_page()	Maps a page into the kernel virtual address space for memcpy access.
kunmap_local()	Releases a kmap_local_page() mapping.
mutex_lock() / mutex_unlock()	Acquires/releases io_mutex to serialize USB operations.
blk_mq_start_request()	Tells blk-mq that processing of the request has started.
blk_mq_end_request()	Completes a block request and reports status to the block layer.
blk_mq_alloc_tag_set()	Allocates the blk-mq hardware queue tag set.
blk_mq_alloc_disk()	Allocates the gendisk and request queue together.
device_add_disk()	Registers the disk with the kernel — creates /dev/myusb0.

13. Testing Procedure

13.1 Build and Load

```
make -C /lib/modules/$(uname -r)/build M=$PWD modules
sudo insmod usb_ms.ko
dmesg | tail -20          # check for "USB Mass Storage driver ready"
```

13.2 Connect and Verify

```
lsusb          # verify device appears
lsusb -t      # show device tree with speed
lsblk         # confirm /dev/myusb0 appears
blkid         # show partition filesystem type
```

13.3 Mount and Access

```
sudo mount /dev/myusb0p2 /mnt/usb
ls /mnt/usb      # list files on device
```

13.4 Write Test

```
sudo dd if=/dev/zero of=/mnt/usb/speedtest.bin bs=1M count=100 oflag=direct
status=progress
```

13.5 Read Test

```
sudo dd if=/mnt/usb/speedtest.bin of=/dev/null bs=4M count=100 status=progress
```

13.6 Unmount and Unload

```
sync
sudo umount /mnt/usb
sudo rmmod usb_ms
dmesg | tail -5          # confirm "MY_USB: driver unloaded"
```

14. Test Results

14.1 Device Identification (lsusb -v)

The lsusb command confirms the device was correctly identified with the expected USB class, subclass, and protocol values:

```
Bus 004 Device 002: ID 8564:4000 Transcend Information, Inc. microSD/SD/CF UHS-II Card Reader [RDF8, RDF9]
Couldn't open device, some information will be missing
Negotiated speed: SuperSpeed (5Gbps)
Device Descriptor:
  bLength                18
  bDescriptorType        1
  bcdUSB                  3.20
  bDeviceClass            0 [unknown]
  bDeviceSubClass         0 [unknown]
  bDeviceProtocol         0
  bMaxPacketSize0         9
  idVendor                0x8564 Transcend Information, Inc.
  idProduct               0x4000 microSD/SD/CF UHS-II Card Reader [RDF8, RDF9]
  bcdDevice               0.09
  iManufacturer          3 TS-RDF5A
  iProduct                4 Transcend
  iSerial                 2 000000000000
  bNumConfigurations     1
Configuration Descriptor:
  bLength                9
  bDescriptorType        2
  wTotalLength           0x002c
  bNumInterfaces         1
  bConfigurationValue    1
  iConfiguration         0
  bmAttributes            0x80
    (Bus Powered)
  MaxPower               896mA
Interface Descriptor:
  bLength                9
  bDescriptorType        4
  bInterfaceNumber       0
  bAlternateSetting      0
  bNumEndpoints          2
  bInterfaceClass        8 Mass Storage
  bInterfaceSubClass     6 SCSI
  bInterfaceProtocol     80 Bulk-Only
  iInterface              0
Endpoint Descriptor:
  bLength                7
  bDescriptorType        5
  bEndpointAddress       0x81  EP 1 IN
  bmAttributes           2
    Transfer Type        Bulk
    Synch Type           None
    Usage Type           Data
  wMaxPacketSize         0x0400  1x 1024 bytes
  bInterval              0
  bMaxBurst              4
Endpoint Descriptor:
  bLength                7
  bDescriptorType        5
  bEndpointAddress       0x02  EP 2 OUT
  bmAttributes           2
    Transfer Type        Bulk
    Synch Type           None
    Usage Type           Data
  wMaxPacketSize         0x0400  1x 1024 bytes
  bInterval              0
  bMaxBurst              4
```

Figure 1: lsusb -v output — Transcend microSD/CF Card Reader, Mass Storage / SCSI / Bulk-Only, SuperSpeed 5 Gbps

14.2 Interface Class Confirmation

The interface descriptor confirms bInterfaceClass = 8 (Mass Storage), bInterfaceSubClass = 6 (SCSI), bInterfaceProtocol = 80 (Bulk-Only):

bInterfaceClass	8 Mass Storage
bInterfaceSubClass	6 SCSI
bInterfaceProtocol	80 Bulk-Only

Figure 2: Interface descriptor showing correct class/subclass/protocol values

14.2 USB Device Detection

```
[92709.191815] my_usb_mass_storage 4-1:1.0: Bulk IN: 0x81
[92709.191825] my_usb_mass_storage 4-1:1.0: Bulk OUT: 0x02
[92709.193188] my_usb_mass_storage 4-1:1.0: CSW signature: 0x53425355
[92709.193198] my_usb_mass_storage 4-1:1.0: CSW tag: 1
[92709.193202] my_usb_mass_storage 4-1:1.0: CSW residue: 0
[92709.193207] my_usb_mass_storage 4-1:1.0: CSW status: 0x00
[92709.193223] my_usb_mass_storage 4-1:1.0: INQUIRY successful
[92709.193226] my_usb_mass_storage 4-1:1.0: Vendor   : TS-RDF5A
[92709.193231] my_usb_mass_storage 4-1:1.0: Product  : Transcend
[92709.193235] my_usb_mass_storage 4-1:1.0: Revision : 0009
[92709.193516] my_usb_mass_storage 4-1:1.0: CSW signature: 0x53425355
[92709.193521] my_usb_mass_storage 4-1:1.0: CSW tag: 2
[92709.193524] my_usb_mass_storage 4-1:1.0: CSW residue: 0
[92709.193527] my_usb_mass_storage 4-1:1.0: CSW status: 0x00
[92709.193861] my_usb_mass_storage 4-1:1.0: CSW signature: 0x53425355
[92709.193865] my_usb_mass_storage 4-1:1.0: CSW tag: 3
[92709.193868] my_usb_mass_storage 4-1:1.0: CSW residue: 0
[92709.193872] my_usb_mass_storage 4-1:1.0: CSW status: 0x00
[92709.193880] my_usb_mass_storage 4-1:1.0: Last LBA: 61951999
[92709.193884] my_usb_mass_storage 4-1:1.0: Block size: 512
[92709.193887] my_usb_mass_storage 4-1:1.0: Capacity: 31719424000 bytes
[92709.197374] my_usb_mass_storage 4-1:1.0: CSW signature: 0x53425355
[92709.197380] my_usb_mass_storage 4-1:1.0: CSW tag: 4
[92709.197383] my_usb_mass_storage 4-1:1.0: CSW residue: 0
[92709.197385] my_usb_mass_storage 4-1:1.0: CSW status: 0x00
[92709.197414] myusb0: p1 p2
[92709.197992] my_usb_mass_storage 4-1:1.0: Block device /dev/myusb0 created
[92709.197996] my_usb_mass_storage 4-1:1.0: USB Mass Storage driver ready
[92709.198117] usbcore: registered new interface driver my_usb_mass_storage
[92709.198122] MY_USB: driver loaded
```

Figure 3: Usb Device detection ,Bulk in and bulk out endpoints, Inquiry, Read capacity, usb core registration .

14.4 Mount and Filesystem Access

The device was successfully mounted and directory listing confirms the filesystem is accessible:

```
[sudo: authenticate] Password:
mirafra@mirafra:~/prudhvi/project$ sudo mount /dev/myusb0p2 /mnt/usb/
mirafra@mirafra:~/prudhvi/project$ cd /mnt/usb/
mirafra@mirafra:/mnt/usb$ ls
bin  dev  home  lost+found  mnt  proc  run  speedtest.bin  sys  usr
boot  etc  lib  media      opt  root  sbin  srv           tmp  var
mirafra@mirafra:/mnt/usb$
```

Figure 4: `sudo mount /dev/myusb0p2 /mnt/usb` — filesystem mounted; standard Linux directory tree visible

14.5 Write Performance

100 MiB written to the device using `dd` with direct I/O (`oflag=direct` bypasses the page cache, measuring true device write speed):

```
mirafra@mirafra:/mnt/usb$ sudo dd if=/dev/zero of=speedtest.bin bs=1M count=100 oflag=direct status=progress
103809024 bytes (104 MB, 99 MiB) copied, 3.01975 s, 34 MB/s
100+0 records in
100+0 records out
104857600 bytes (105 MB, 100 MiB) copied, 3.04647 s, 34 MB/s
```

Figure 5.1: Write test — 100 MiB written at 34 MB/s in ~3.05 seconds

14.6 Read Performance

400 MiB read back from the device. The higher read speed is expected as USB bulk IN transfers from flash media are generally faster than writes:

```
mirafra@mirafra:~$ sudo dd if=/mnt/usb/speedtest.bin of=/dev/null bs=4M count=100 status=progress
390070272 bytes (390 MB, 372 MiB) copied, 4.02854 s, 97 MB/s
100+0 records in
100+0 records out
419430400 bytes (419 MB, 400 MiB) copied, 4.33073 s, 97 MB/s
```

Figure 5.2: Read test — 400 MiB read at 97 MB/s in ~4.33 seconds

14.7 Performance Summary

Metric	Result
Write Speed	34 MB/s (100 MiB, <code>dd</code> + <code>oflag=direct</code>)
Read Speed	97 MB/s (400 MiB, <code>dd</code> from device)
Device	Transcend microSD/CF Card Reader (USB 3.2 Gen 1)
Block Size	512 bytes (confirmed via READ CAPACITY)
Mount Result	Successful — full Linux filesystem visible

15. Conclusion

This project successfully implements a fully functional USB Mass Storage device driver as a Linux kernel module. The driver correctly follows the USB Bulk-Only Transport specification, implements all required SCSI commands, and exposes the device as a standard Linux block device.

The driver was tested on real hardware (a Transcend USB card reader running at SuperSpeed 5 Gbps) and achieved write speeds of 34 MB/s and read speeds of 97 MB/s. The device was mounted, read, and written using standard Linux commands without any kernel instability.

All major components were implemented from scratch: endpoint discovery, CBW/CSW construction, SCSI command encoding, blk-mq block device registration, page-mapped I/O transfer, error recovery, and clean disconnect handling.